BAHRIA UNIVERSITY
Discovering Knowledge

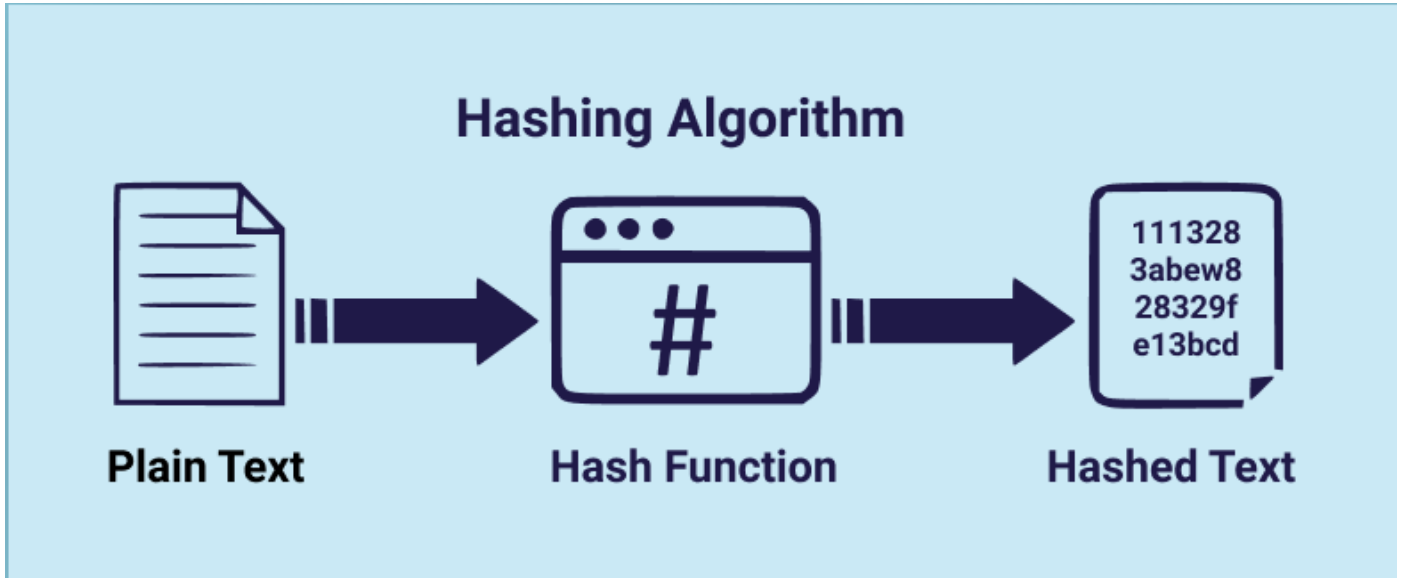
Cryptographic Hash Functions

Data Encryption & Security (CEN-451)

Spring 2025 (BSE-8A&B)

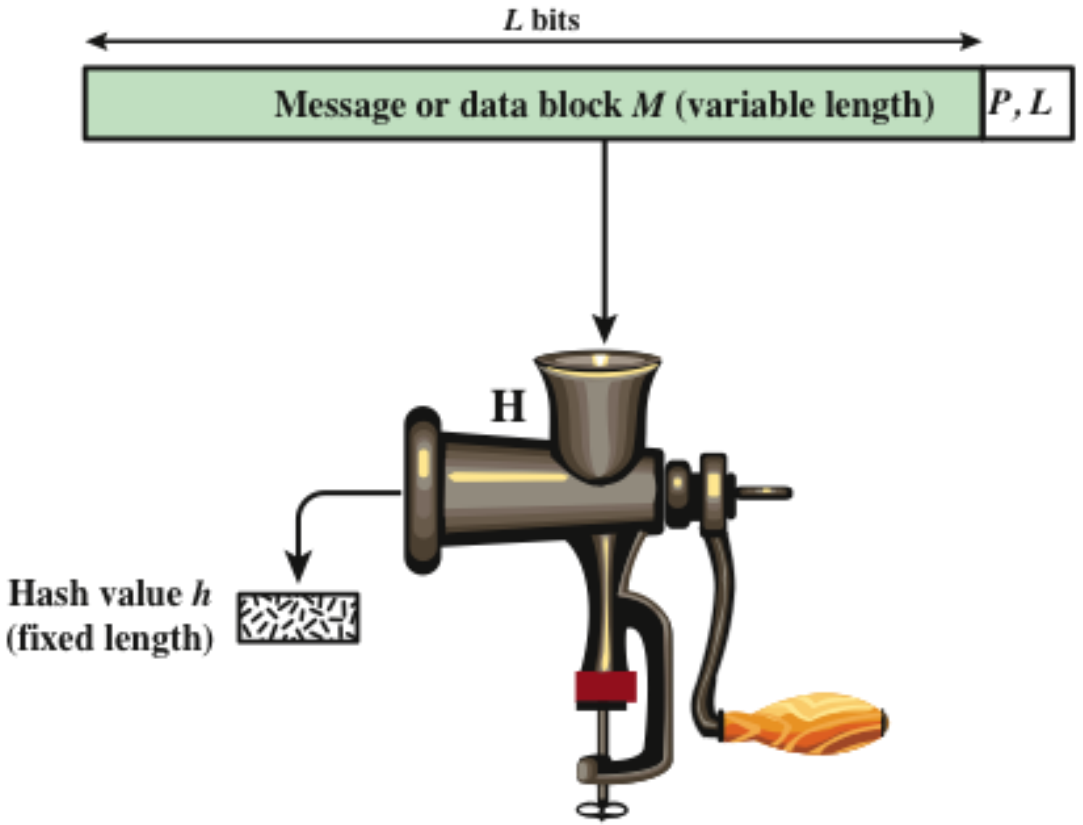
Hash Functions: An Overview

- The principal objective of a hash function is **data integrity**.
- Hash functions are often used to determine whether or not data has been **changed**.



Hash Functions & Hash Value

- A hash function **H** accepts a **variable-length** block of data **M** as input and produces a **fixed-size** hash value ***h = H(M)***.
- Typically, the input is padded out to an **integer multiple** of some fixed length **(e.g. 1024 bits)**.
- A change to any bit or bits in ***M*** results with high probability in a change to the hash value ***h***.



P, L = padding plus length field

Hash Function Algorithms

Algorithm	Message Block (Bits)	Hash Value (h) (Bits)
SHA-0	512	160
SHA-1	512	160
SHA-224/256	512	224/256
SHA-384/512	1024	384/512
MD5	512	128
Whirlpool	512	512

Hash Functions Requirements

- The algorithms for **cryptographic hash function** should satisfy the following two conditions:

1. It should be **computationally** infeasible to find a data that maps to a pre-specified hash value **(the one-way property)**.

2. It should be **computationally** infeasible to find two data that map to the same hash value **(the collision-free property)**.

Applications of Hash Functions

- Hash functions can be used for several applications including:
  1. Message authentication.
  2. One-way password file **(used by most OS)**
  3. File intrusion detection.
  4. Virus detection.

Message Authentication

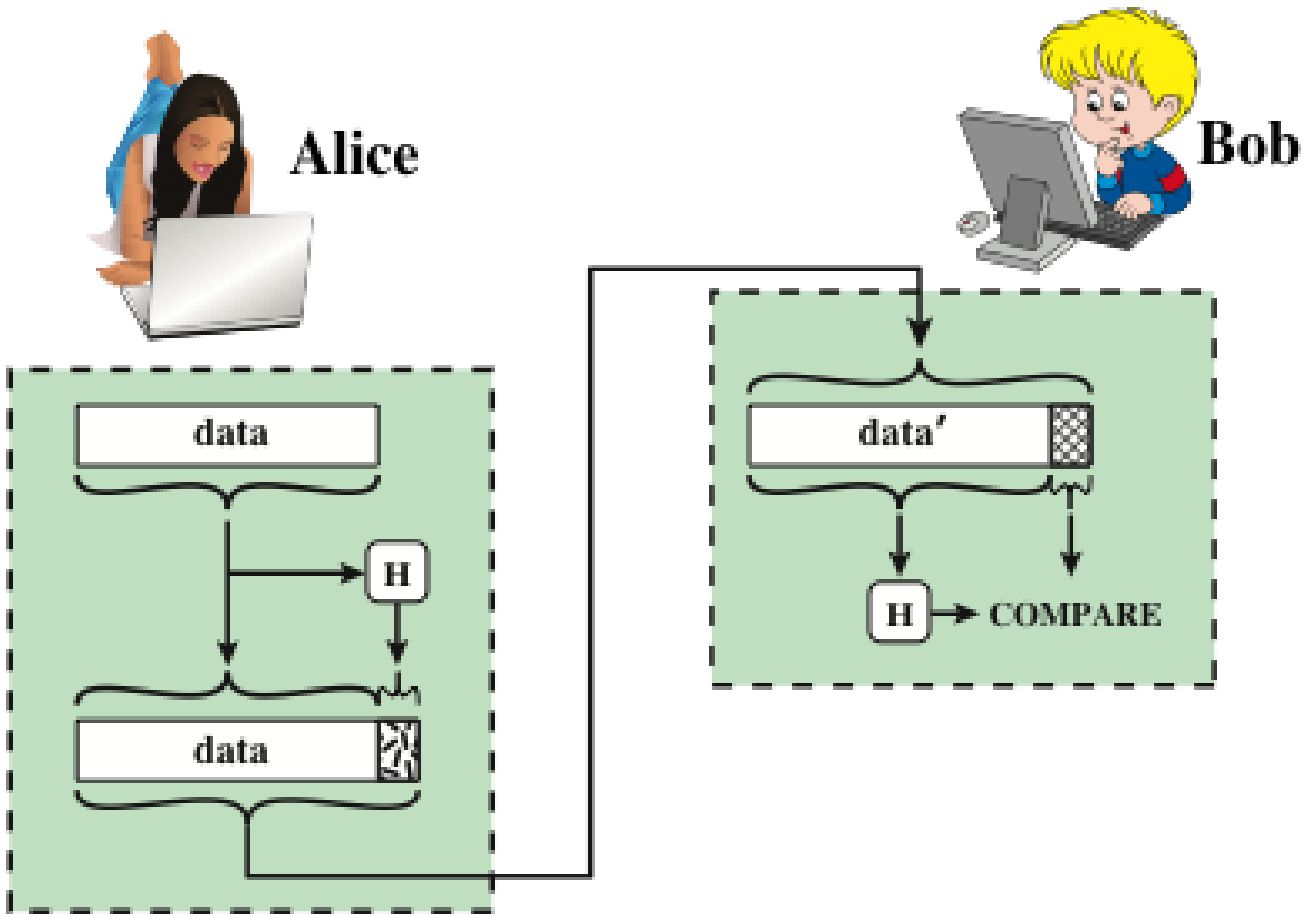
- Message authentication assures that data received are exactly as sent.
- Hence, message authentication is a mechanism used to verify the **integrity of a message**.
- When hash function is used for message authentication, the hash function value is often referred to as a **message digest**.

Message Authentication (Cont.)

Hash function usage mechanism for message authentication:

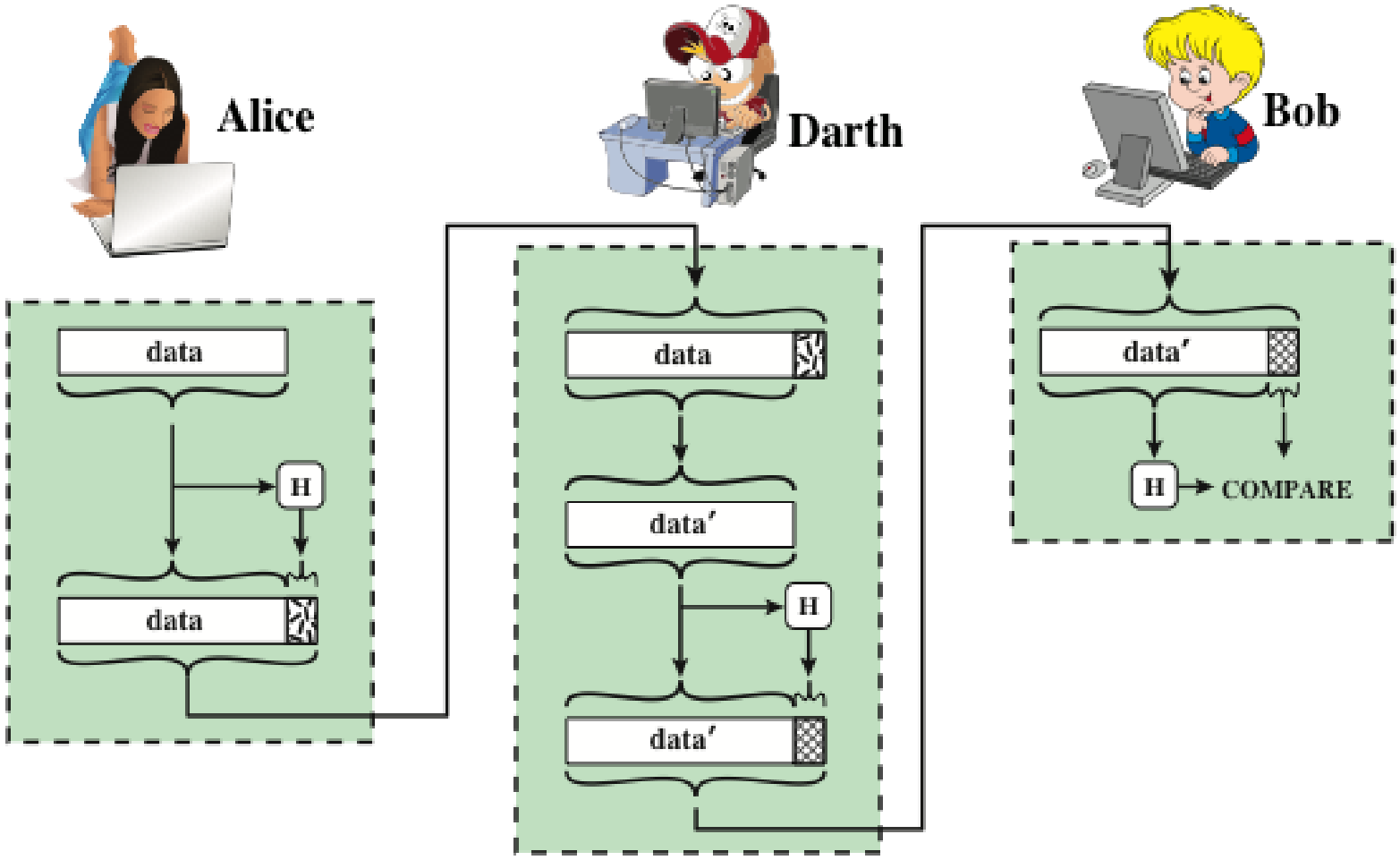
- Sender computes a **hash value** as a function of bits in message.
- Sender transmits both **hash value** and message.
- Receiver performs the same hash calculation on received message and compares this value with incoming **hash value**.
- If mismatch, receiver knows that message or **hash value** has been altered.

Message Authentication (Cont.)



(a) Use of hash function to check data integrity

Message Authentication (Cont.)



(b) Man-in-the-middle attack

Message Authentication (Cont.)

Q) Why hash value must be transmitted in a secure and protected manner?

A)

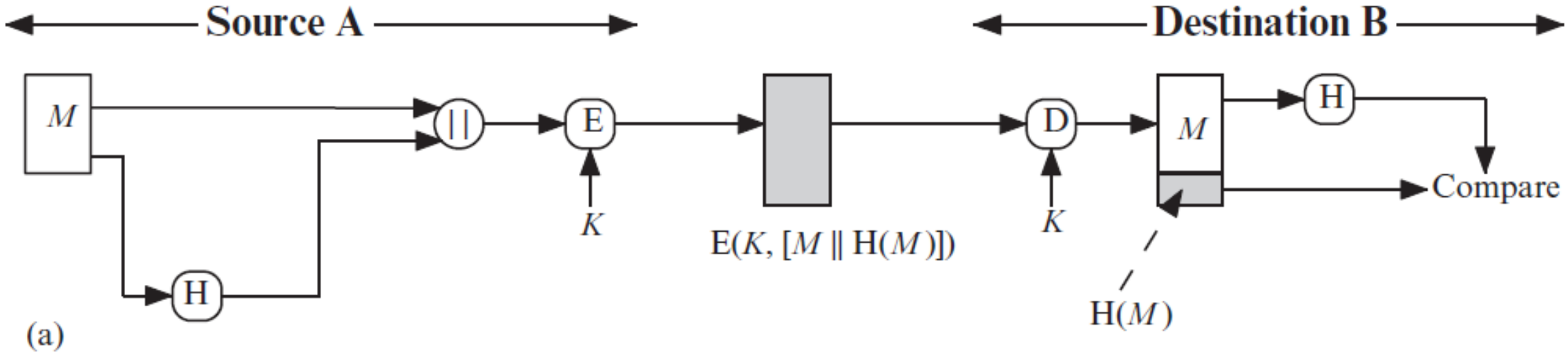
- If an adversary alters the message, it should not be feasible for him/her to also alter the hash value to fool the receiver.
- To prevent an attack known as the **Man-in-the-Middle (MITM)** attack, the hash value must be protected.

Note: there are variety of ways in which a hash code can be used to provide message authentication ***(explained in next slides)***.

Message Authentication (Cont.)

Usage-I:

- The message & concatenated hash code are encrypted using **symmetric encryption**.



Message Authentication (Cont.)

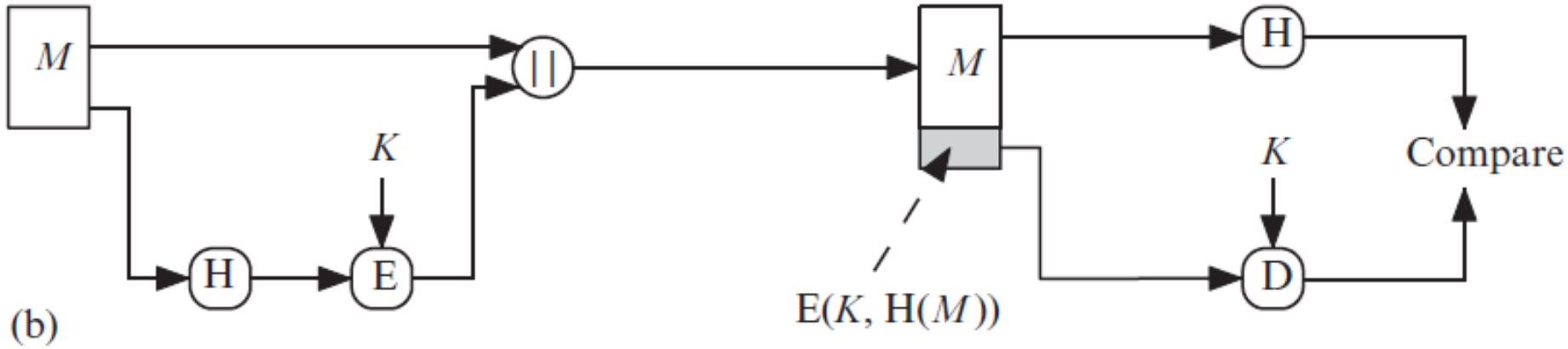
Usage-I (Cont.):

- Due to encryption using a secret key, message must have come from authorized user, hence **user authentication** is obtained.
- Due to hash code, **message authentication** is obtained.
- Due to encryption applied to entire message plus hash code, **message confidentiality** is obtained.

Message Authentication (Cont.)

Usage-II:

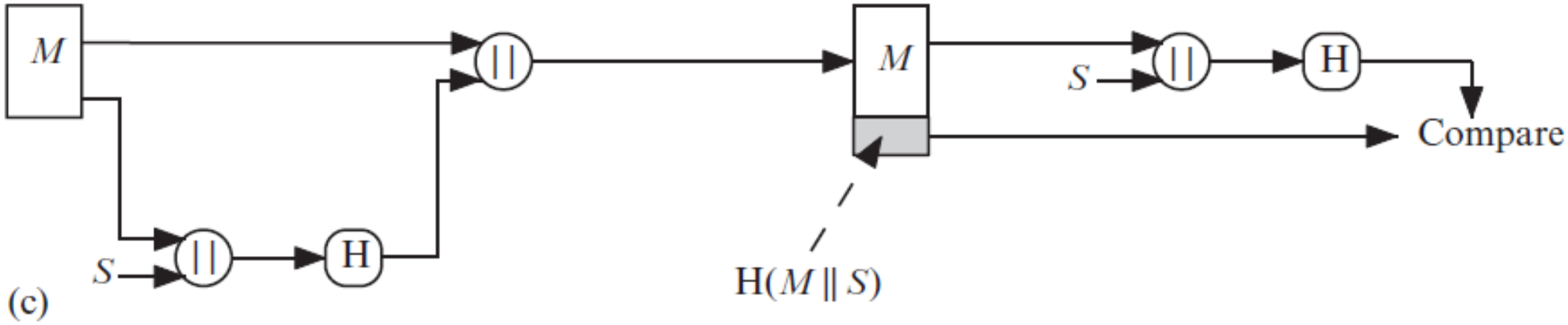
- Only hash code is encrypted, using **symmetric encryption**.
- Reduces the processing burden for those applications that do not require confidentiality.



Message Authentication (Cont.)

Usage-III:

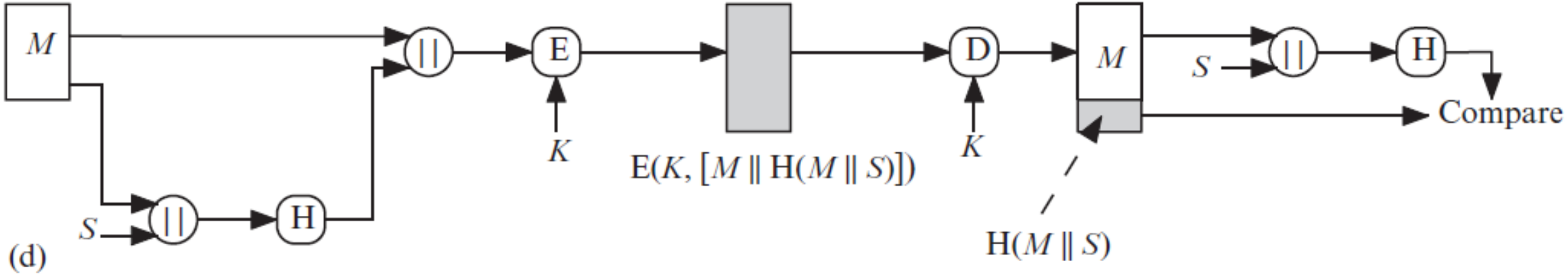
- Assumes that the two communicating parties share a common secret value **S**.
- Because the secret value is not sent, an opponent cannot modify an intercepted message and cannot generate a false message.



Message Authentication (Cont.)

Usage-IV:

- Confidentiality can be added to the approach of **Method-III** by encrypting the entire message plus the hash code.

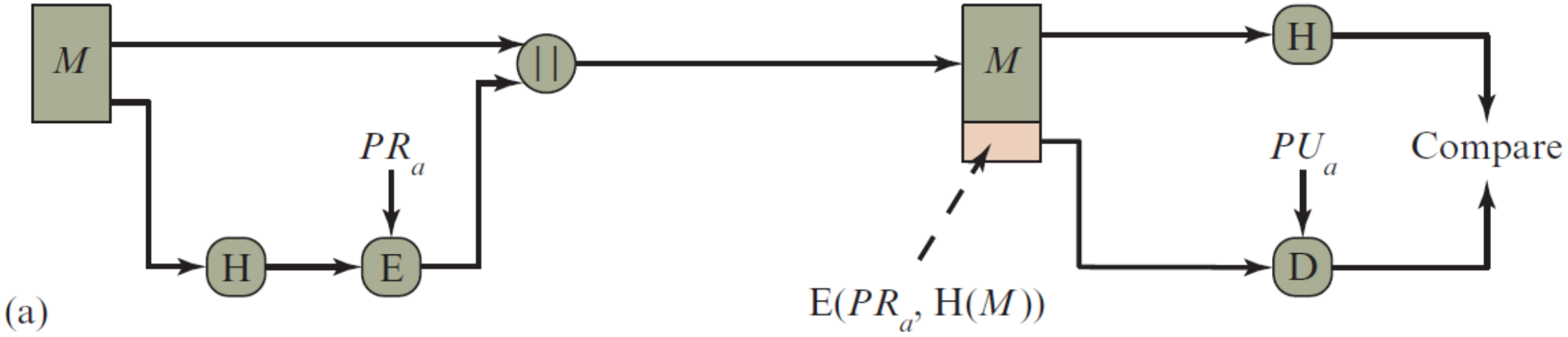


Digital Signatures

- To achieve **digital signature**, the hash value of a message is encrypted with a user's **private key**.
- Anyone who knows the user's **public key** can verify **integrity of message** that is associated with the **digital signature**.
- An attacker who wishes to alter the message would need to know the user's **private key**.

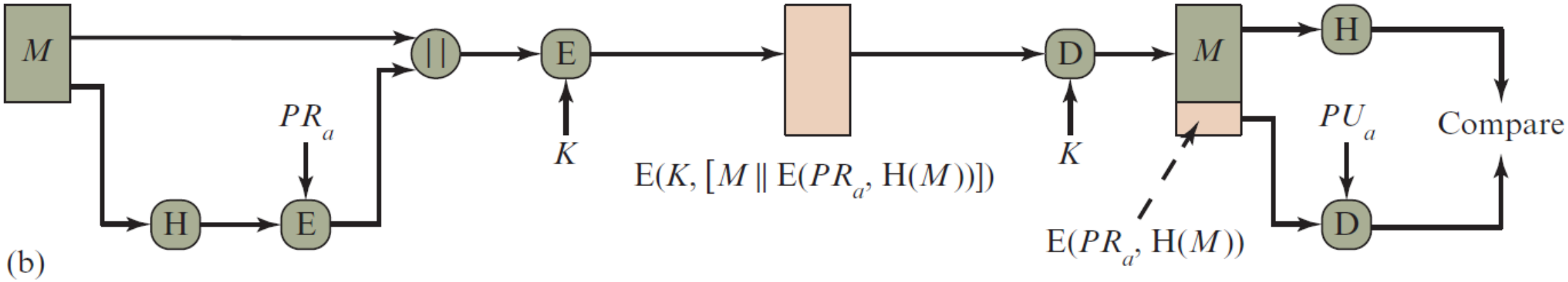
Digital Signatures (Cont.)

- Hash code is encrypted with sender's **private key**.
- This provides **authentication** and **digital signature**.



Digital Signatures (Cont.)

- If both **confidentiality** and **digital signature** is desired, then the ***message*** plus ***private-key-encrypted hash code*** can be encrypted using a **symmetric secret key**.



One-way Password File

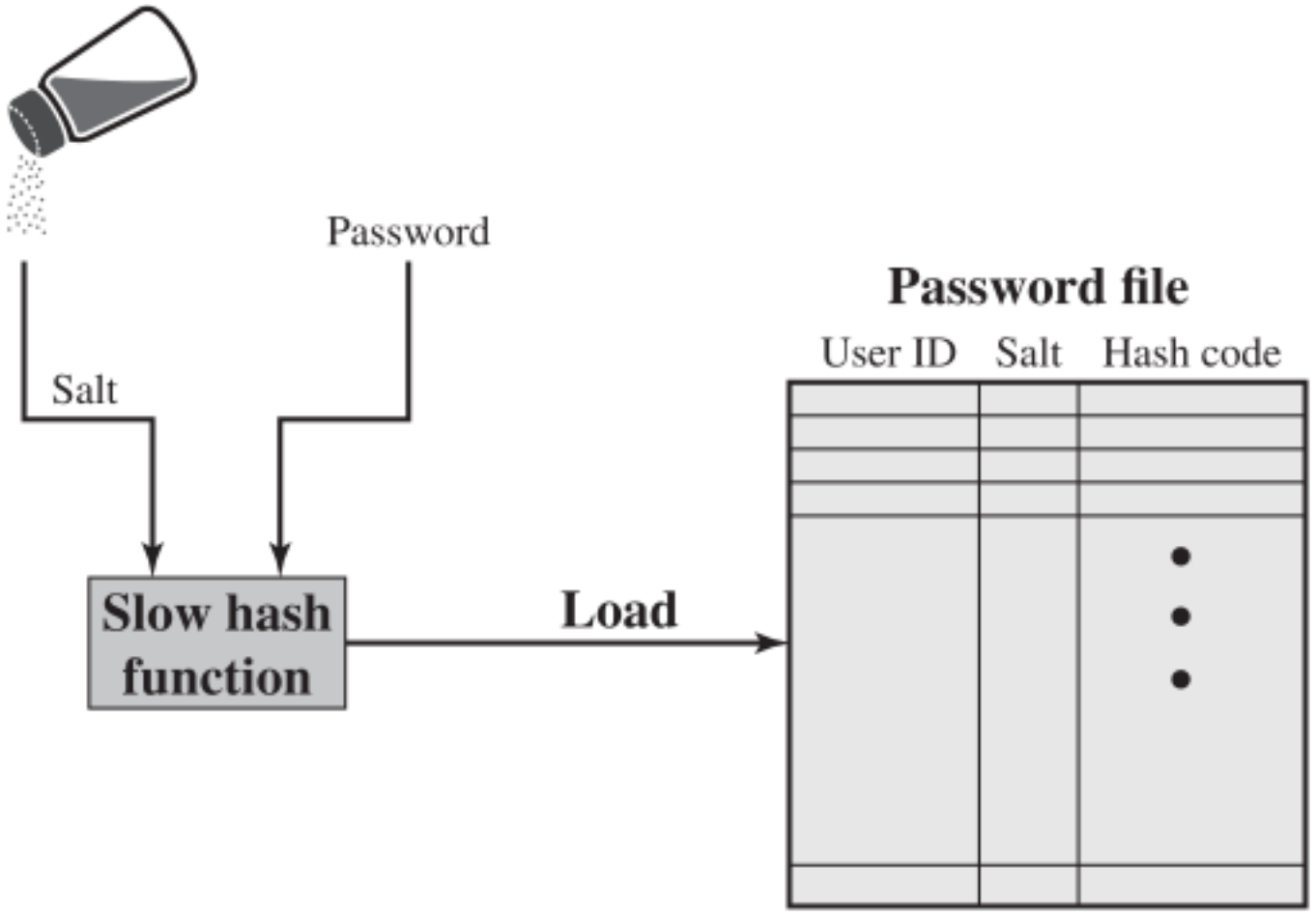
- In general, a **hash** of a **password** is stored by an OS rather than the **password** itself.
- The actual **password** is not retrievable by a hacker who gains access to the **password file**.
- When a user enters a **password**, the **hash** of that **password** is compared to the stored **hash** value for verification.
- Above approach to **password** protection is used by most OS.

Hashed Passwords

- A widely used password security technique is the use of **hashed passwords** and a **salt value**. Such scheme is found on **UNIX** variants and on a number of other OS.
- This password is combined with a fixed-length **salt value** which is a **random number**.
- The **password** and **salt value** serve as inputs to a hashing algorithm to produce a fixed-length **hash code**.
- The hash algorithm is designed to be ***"slow to execute"*** in order to prevent attacks (e.g. **brute force attacks**).

Hashed Passwords (Cont.)

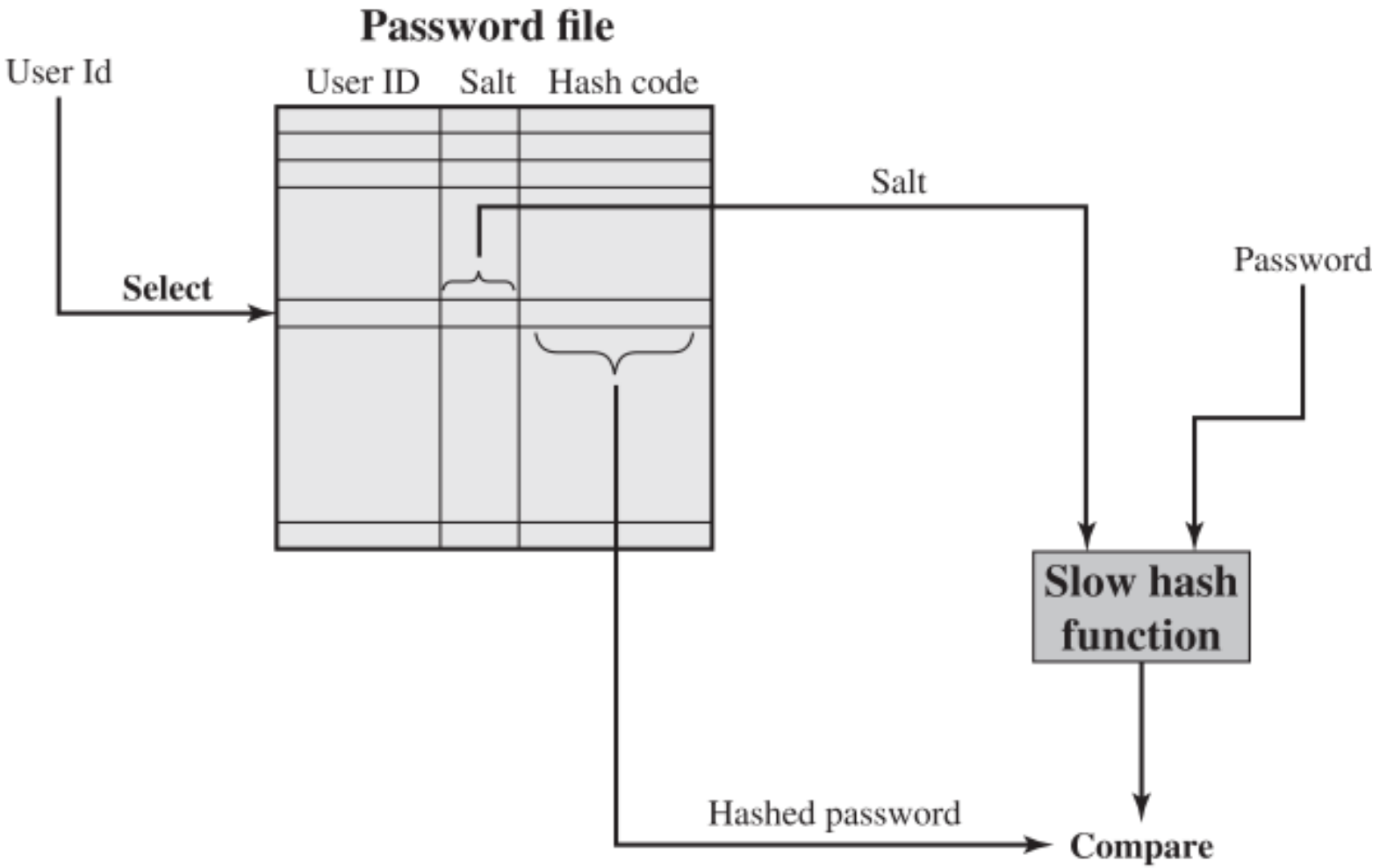
- The **Hashed password** is stored together with a plaintext copy of the **salt value** in the **"password file"** for the corresponding user ID.



Hashed Passwords (Cont.)

- When a user attempts to logon to the OS, the user provides an ID and a password.
- The OS uses the **ID to index** into the password file and retrieve the plaintext **salt value** and the **hashed password**.
- The **salt value** and user-supplied password are used as input to the hash fun, and if the result matches the stored value, the password is accepted.

Hashed Passwords (Cont.)



Salt Value's Purpose

1. Prevents **duplicate passwords** from being visible in the password file. Even if two users choose the same password, those passwords will be assigned different **salt values**. Hence, the hashed passwords of the two users will differ.
2. It becomes nearly impossible to find out whether a person with passwords on ***two or more systems*** has used the same password on all of them.
3. Increases the difficulty of **off-line dictionary attacks**.

File Intrusion Detection

- Discussed in Class.
- Mechanism is part of course.

Virus Detection

- Discussed in Class.
- Mechanism is part of course.

Simple Hash Function

- All hash functions operate using the following principles:
  - The ***input (message, file, etc.)*** is viewed as a sequence of ***n* - bit** blocks.
  - The ***input*** is processed one block at a time in an **iterative fashion** to produce a hash.
- Two simple, but **insecure** hash functions, are presented in subsequent slides.

Simple Hash Function: XOR

- One of the simplest hash functions is the **bit-by-bit XOR** of every block, expressed as:

$$C_i = b_{i1} \oplus b_{i2} \oplus \cdots \oplus b_{im}$$

where

C_i = ith bit of the hash code, $1 \leq i \leq$ n

m = number of n-bit blocks in the input

b_{ij} = ith bit in jth block

$\oplus$ = XOR operation

Simple Hash Function: XOR (Cont.)

	bit 1	bit 2	• • •	bit n
Block 1	b_{11}	b_{21}		b_{n1}
Block 2	b_{12}	b_{22}		b_{n2}
	•	•	•	•
	•	•	•	•
	•	•	•	•
Block m	b_{1m}	b_{2m}		b_{nm}
Hash code	C_1	C_2		C_n

Simple Hash Function: XOR (Cont.)

- **Example:** for the input given below and **n = 8**

  00111011 11101101 00101000 00101011 01011000 11001110

- The output using XOR operation is:

00111011	**(1st input block)**
11101101	**(2nd input block)**
00101000	**(3rd input block)**
00101011	**(4th input block)**
01011000	**(5th input block)**
11001110	**(6th input block)**
01010011	**(Hash Value)**

Simple Hash Function: XOR (Cont.)

- This operation produces a **simple parity bit** for each bit position and is known as a **longitudinal redundancy check**.
- It is **reasonably effective** as a data integrity check.
- The probability that a **data error** will result in an **unchanged hash value** is 2^{-n}.
- However, the function is less effective with more **predictably formatted data**.

Simple Hash Function: RXOR

- A simple way to improve security is to perform a **one-bit circular shift**, on the hash value after **each block** is processed.
- The procedure can be summarized as follows:
  1. Initially set the **n-bit** hash value to **zero**.
  2. Process each successive **n-bit** block of data as follows:
     - Rotate the current **hash value** to the **left / right** by **one bit**
     - XOR the block into the hash value
- RXOR method has the effect of **randomizing** to overcome any **regularities** that appear in the input.

Simple Hash Function: RXOR (Cont.)

- **Example:** for input below, **n = 8** and **Left-Shift**

  00111011 11101101 00101000 00101011 01011000 11001110

- The output using RXOR operation is:

00000000 (Initial Hash and left-shift)
00111011 (1st Block)
00111011 (1st XOR Hash)

01110110 (Hash left-shift)
11101101 (2nd Block)
10011011 (2nd XOR Hash)

Simple Hash Function: RXOR (Cont.)

00110111	(Hash left-shift)
00101000	(3rd Block)
00011111	(3rd XOR Hash)
00111110	(Hash left-shift)
00101011	(4th Block)
00010101	(4th XOR Hash)
00101010	(Hash left-shift)
01011000	(5th Block)
01110010	(5th XOR Hash)

Simple Hash Function: RXOR (Cont.)

11100100 **(Hash left-shift)**
11001110 **(6th Block)**
00101010 **(6th and final XOR Hash)**

00101010 **(Hash value h)**

Thank You!